Azure POC 04 - Document Intelligence ETL Pipeline

Beginner End-to-End Implementation, Verification and Troubleshooting Guide

Contents

What This POC Proves

Business goal

Convert unstructured synthetic invoices into validated analytical records without using real financial documents.

- Upload invoice PDFs/images to `documents/incoming/`.
- Extract invoice fields and line items using the **prebuilt invoice** model.
- Validate required fields, confidence scores and financial reconciliation.
- Write valid headers/lines into Azure SQL.
- Write invalid documents to the Blob `failed/` quarantine path with reasons.
- Prevent duplicates by SHA-256 based idempotency checks.

Completion conditions

- At least five invoices are processed successfully.
- One malformed invoice is rejected/quarantined.
- Reprocessing the same input does not duplicate SQL rows.
- Header totals reconcile with line amounts where extraction is complete.
- No secrets are committed to Git and no real documents are used.

> **Version clarification**
>
> The final project version creates `dbo.invoice_header` and `dbo.invoice_line`. It does not create a `processing_audit_log` table; quarantine details are stored as JSON under `documents/failed/`.

Architecture and Data Flow

1. Synthetic invoices
samples/input/*.pdf

->

2. ADLS Gen2
documents/incoming/

->

3. Document Intelligence
prebuilt-invoice

->

4. Python validation
confidence + totals

VALID

Azure SQL

dbo.invoice_header

dbo.invoice_line

+

documents/processed/*.json

INVALID / EXCEPTION

documents/failed/*.failure.json

failure_reason

warnings

normalized payload when available

Beginner note

Blob "folders" are prefixes. The code creates `incoming/`, `processed/`, `processed/raw/`, and `failed/` as files are uploaded; you do not need to pre-create physical directories.

Project Structure - What Each File Does

```
POC_04_DOCUMENT_INTELLIGENCE_ETL_PROJECT/
  .env.example
  requirements.txt
  function_app.py
  host.json
  src/
    config.py
    storage_client.py
    extract_invoice.py
    validate_invoice.py
    sql_loader.py
    pipeline.py
    upload_samples.py
    run_batch.py
  samples/input/
    invoice_001.pdf ... invoice_005.pdf
    invoice_006_malformed.pdf
  scripts/
    generate_sample_invoices.py
    01_create_azure_resources.ps1
    02_show_resource_values.ps1
    99_cleanup.ps1
  sql/
    create_tables.sql
    create_function_identity_user.sql
    verify.sql
  schemas/
    invoice_schema.json
  docs/
    portal_manual_steps.md
    confidence_rules.md
    troubleshooting.md
  tests/
    test_validation.py
```

Important

Run commands from the project root. The `src` folder is a Python package, so use module execution such as `python -m src.upload_samples` rather than `python src\upload_samples.py`.

Recommended Beginner Execution Order

1. Check Python and Azure CLI
2. Create/activate virtual environment
3. Install requirements and run tests
4. Review synthetic invoices
5. Create Azure resources
6. Test one invoice manually in Document Intelligence
7. Retrieve Azure resource values
8. Create local `.env`
9. Create Azure SQL tables
10. Upload sample invoices
11. Run batch ETL
12. Verify SQL + Blob outputs + duplicates
13. Optionally deploy Function App
14. Delete Azure resources after the POC

Important

Do not jump directly to `run_batch` before creating the SQL schema when `ENABLE_SQL_LOAD=true`. That exact ordering mistake caused the main POC error.

Prerequisites on Windows

Check Python

```powershell
python --version
```

Recommended project environment: Python 3.12.

Check Azure CLI

```powershell
az version
az login
az account show
```

If you have multiple subscriptions

```powershell
az account list -o table
az account set --subscription
"<SUBSCRIPTION_NAME_OR_ID>"
```

Permissions

- Ability to create resources in your Azure subscription.
- Data-plane RBAC for Blob access.
- Permission to configure Azure SQL Microsoft Entra administration / database objects.

> **Common prerequisite issue**
>
> If `az` is not recognized, install Azure CLI, close/reopen the terminal, then retry `az version`.

Local Python Environment

Open Command Prompt or PowerShell in the project root, then create and prepare the environment.

POWERSHELL

```
python -m venv .venv

# PowerShell
.\.venv\Scripts\Activate.ps1

# OR Command Prompt
.venv\Scripts\activate.bat

python -m pip install --upgrade pip
pip install -r requirements.txt
```

Beginner note

If PowerShell blocks activation, use Command Prompt and `.venv\Scripts\activate.bat`, or temporarily adjust the PowerShell execution policy for the current process.

Sanity check

COMMAND

```
python -c "import azure.identity; import azure.storage.blob; print('Python dependencies import successfully')"
```

Validate the Python Rules Before Azure

Run the validation tests before provisioning or debugging cloud resources:

POWERSHELL

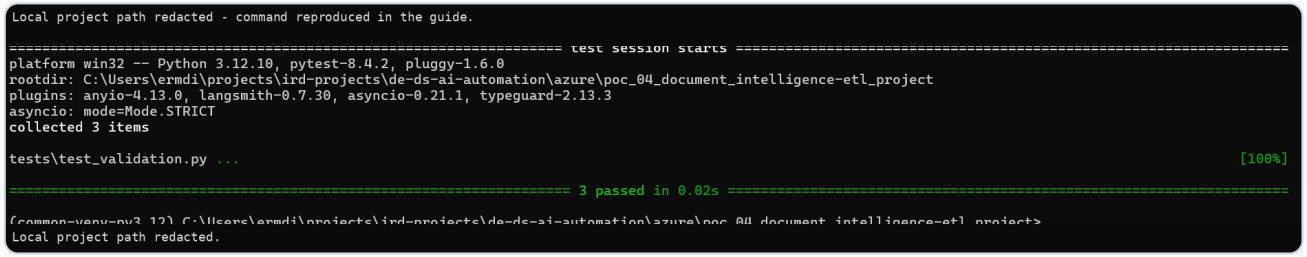
```
python -m pytest tests/test_validation.py

# Short form
python -m pytest -q
```

Expected result

Expected: **3 passed**. This verifies required-field, amount-reconciliation and confidence-threshold behavior.

Fix for import-path error

If standalone `pytest tests/test_validation.py` raises `ModuleNotFoundError: No module named src`, use `python -m pytest ...` or add a `pytest.ini` containing `[pytest] pythonpath = .`.



Successful validation test run. Local user path has been redacted.

Synthetic Invoice Inputs

Included test documents

- `invoice_001.pdf` through `invoice_005.pdf` - intended success cases.
- `invoice_006_malformed.pdf` - intentionally malformed for failure/quarantine testing.

Regenerate samples

COMMAND

```
python scripts/generate_sample_invoices.py
```

Fields exercised

- invoice_number
- invoice_date
- supplier_name
- customer_name
- currency
- line_items[]
- subtotal
- tax
- total

Data safety

Use only fictional/synthetic invoice data. The POC requirement explicitly avoids real financial documents.

Azure Resources Used by the POC

Resource	Purpose	Typical POC name
Resource Group	Lifecycle boundary for the entire POC	`rg-poc04-docintel`
ADLS Gen2 Storage	Incoming, raw, processed and failed documents	`stpoc04di<suffix>`
Document Intelligence	Invoice field + line extraction	`di-poc04-<suffix>`
Key Vault	Store DI key for local beginner authentication	`kv-poc04-<suffix>`
Azure SQL	Curated invoice header and line records	`sql-poc04-<suffix>` / `sqldb-poc04`
Function runtime storage	Azure Function runtime requirements	`stpoc04fn<suffix>`
Function App + App Insights	Optional event-driven production-style execution + logs	`func-poc04-di-<suffix>`

Cost guardrail

Use the lowest appropriate POC tiers and delete the Resource Group after validation to avoid ongoing cost.

Create Azure Resources - Automated Script

The project includes a PowerShell + Azure CLI provisioning helper. Run it from the project root:

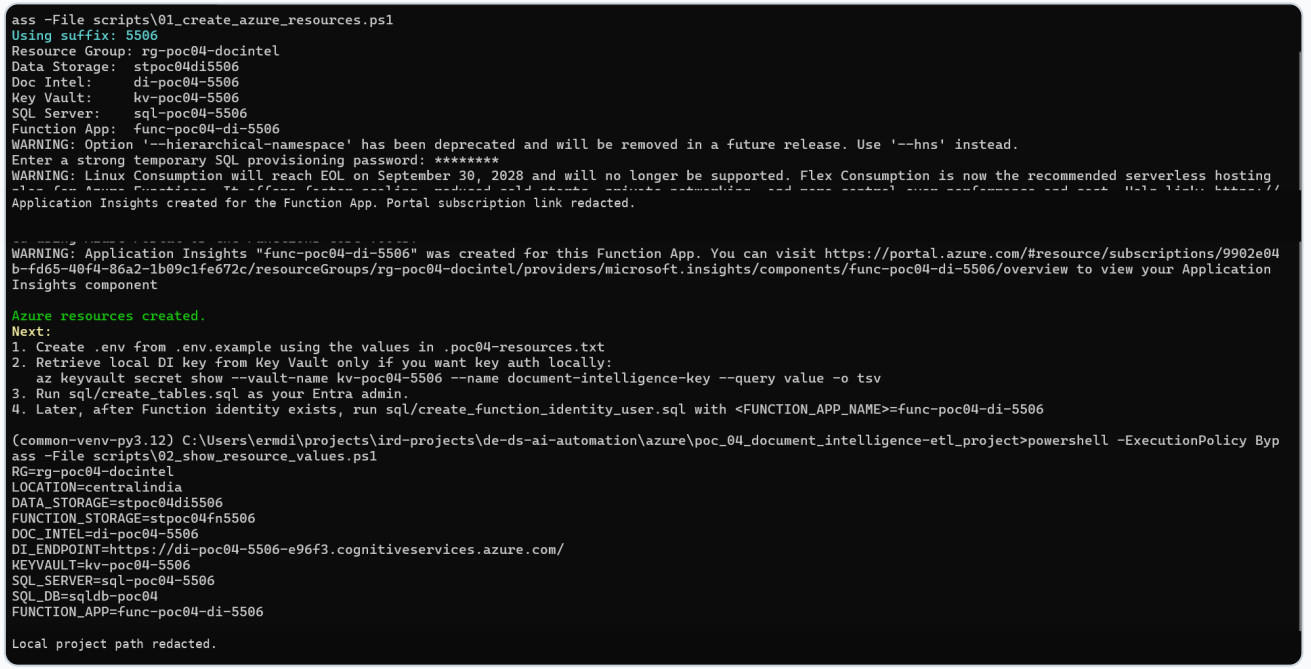
```powershell
az login
powershell -ExecutionPolicy Bypass -File scripts\01_create_azure_resources.ps1
```

When prompted, enter a strong temporary SQL provisioning password and keep it private.

Then display the generated non-secret resource names:

```powershell
powershell -ExecutionPolicy Bypass -File scripts\02_show_resource_values.ps1
```

> **Beginner note**
>
> The script writes resource names/endpoints into `.poc04-resources.txt`; it intentionally does not store passwords/keys there.



Resource provisioning completed. User path and subscription link have been redacted.

Provisioning Warnings Seen During This POC

Warning 1 - CLI flag deprecation

The provisioning output warned that the `--hierarchical-namespace` option is deprecated and that `--hns` should be used instead in a future script update.

Warning 2 - Function hosting plan

The terminal showed a Linux Consumption end-of-life warning and suggested a newer serverless hosting option for future deployments.

Warning 3 - Function not active yet

The Function App resource was created, but the terminal noted it would not become active until function code is published with Azure Functions Core Tools or another deployment method.

> **Outcome in this completed POC**
>
> These were warnings, not blockers. The resource creation script still finished with **Azure resources created**. For a future rerun, review the script flags and hosting plan before provisioning.

Manual Azure Setup - Resource Group

Portal steps

- Open Azure Portal and search **Resource groups**.
- Choose **Create**.
- Select your subscription.
- Name the group `rg-poc04-docintel`.
- Choose a nearby region where the required services are available.
- Review and create.

Verify

- Open the new Resource Group.
- Confirm it is active and initially empty.
- Use this same Resource Group for all remaining POC resources so cleanup is simple.

> **Beginner note**
>
> The automated script uses `centralindia` in the completed project version. If creating manually, keep services in a compatible nearby region where possible.

Manual Azure Setup - ADLS Gen2 and Blob Paths

Create the storage account

- Search **Storage accounts** -> **Create**.
- Resource Group: `rg-poc04-docintel`.
- Use a globally unique lowercase name such as `stpoc04di1234`.
- Performance: Standard; redundancy: LRS for the POC.
- In Advanced settings, enable **Hierarchical namespace** to make it ADLS Gen2 capable.
- Disable public blob access where offered.

Create the container

- Open the storage account -> Containers.
- Create `documents`.
- Assign your signed-in user **Storage Blob Data Contributor** for local execution.

```
documents/
  incoming/
  processed/
    raw/
  failed/
```

> **Beginner note**
>
> The prefixes are created automatically when the Python code uploads files; only the `documents` container must exist.

Manual Azure Setup - Azure AI Document Intelligence

Create the resource

- Search for **Document Intelligence**.
- Create it in the POC Resource Group.
- Use a unique name such as `di-poc04-<suffix>`.
- Use F0 if it is available for your subscription/region; otherwise review cost before choosing a paid tier.
- After creation, open **Keys and Endpoint** and note the endpoint.

Manual first test

- Open Document Intelligence Studio/current Analyze experience.
- Choose the **Invoice / prebuilt invoice** model.
- Upload `samples/input/invoice_001.pdf`.
- Run analysis.
- Check Invoice ID, Invoice Date, Vendor, Customer, Items, SubTotal, TotalTax, InvoiceTotal and confidence scores.

> **Expected result**
>
> Pass condition: you see structured invoice fields, not only OCR text.

Manual Azure Setup - Key Vault

Create and configure

- Search **Key vaults** -> Create.
- Use the POC Resource Group and a unique name such as `kv-poc04-<suffix>`.
- Prefer the Azure RBAC permission model.
- Grant yourself **Key Vault Secrets Officer** if required.
- For the beginner local path, store the Document Intelligence key as secret `document-intelligence-key`.

Retrieve the key only when needed locally

POWERSHELL

```
az keyvault secret show --vault-name kv-poc04-<SUFFIX> --name document-intelligence-key --query value
-o tsv
```

Secret handling

Never paste the real key into documentation, screenshots, source code or Git. Keep it only in the local `.env`. If a key was exposed during learning/debugging, rotate/regenerate it before reuse.

Manual Azure Setup - Azure SQL

Create SQL resources

- Search **SQL databases** -> Create.
- Create/select a logical SQL server.
- Database name: `sqldb-poc04`.
- Choose a low-cost POC-appropriate tier and review estimated cost.
- Configure your signed-in identity as the SQL Server Microsoft Entra administrator.

Networking and schema

- Allow your current public client IP for local testing.
- Open Query Editor, SSMS, Azure Data Studio, or another SQL client.
- Run `sql/create_tables.sql` before the ETL batch if SQL loading is enabled.

> **Schema names to remember**
>
> Final schema tables are `dbo.invoice_header` and `dbo.invoice_line`.

Manual Azure Setup - Function App and Application Insights

Create after local batch succeeds

- Create a Function App using Python.
- Match the Function runtime to a supported Python 3.x version.
- Enable Application Insights.
- Enable the system-assigned Managed Identity.

Managed Identity permissions

- ADLS: **Storage Blob Data Owner** and **Storage Queue Data Contributor** for the trigger connection in this POC.
- Document Intelligence: **Cognitive Services User**.
- Azure SQL: run `sql/create_function_identity_user.sql` after replacing the Function App placeholder.

> **Beginner note**
>
> This is optional for the learning POC. First prove the local `src.run_batch` path works end-to-end.

Retrieve Resource Values for .env

First display the names/endpoints saved by the creation script:

```powershell
powershell -ExecutionPolicy Bypass -File scripts\02_show_resource_values.ps1
```

Retrieve the DI key only for local key authentication:

```powershell
az keyvault secret show --vault-name kv-poc04-<SUFFIX> --name document-intelligence-key --query value -o tsv
```

An earlier debugging step also retrieved a Storage connection string. The final project code does **not** require it; storage uses account name + `DefaultAzureCredential`.

Key/connection-string outputs and local path are redacted.

> **Sanitized screenshot**
>
> The screenshot is retained for reference, but secret outputs have been redacted.

.env - Where Every Value Comes From

Variable	Value source	Example / rule
`AZURE_STORAGE_ACCOUNT_NAME`	`.poc04-resources.txt` / script output	`stpoc04di<suffix>`
`AZURE_STORAGE_CONTAINER`	Project convention	`documents`
`DOCUMENTINTELLIGENCE_ENDPOINT`	`DI_ENDPOINT` from resource output	`https://di-poc04-<suffix>-.../`
`DOCUMENTINTELLIGENCE_API_KEY`	Key Vault secret for local beginner path	Keep private; never commit
`AZURE_SQL_CONNECTIONSTRING`	Construct from SQL server + DB	`Authentication=ActiveDirectoryDefault`
`CRITICAL_CONFIDENCE_THRESHOLD`	Project default	`0.70`
`AMOUNT_TOLERANCE`	Project default	`0.05`
`ENABLE_SQL_LOAD`	Project control	`true`

Beginner note

Run `az login` before local storage/SQL passwordless operations because the project uses Azure identity-based credentials.

.env - Safe Template for Local Execution

Create the file from the template:

COMMAND PROMPT

```
copy .env.example .env
notepad .env
```

DOTENV

```
# Azure Storage
AZURE_STORAGE_ACCOUNT_NAME=stpoc04di<YOUR_SUFFIX>
AZURE_STORAGE_CONTAINER=documents

# Azure AI Document Intelligence
DOCUMENTINTELLIGENCE_ENDPOINT=https://di-poc04-<YOUR_SUFFIX>-<subdomain>.cognitiveservices.azure.com/
DOCUMENTINTELLIGENCE_API_KEY=<PASTE_LOCAL_KEY_HERE>

# Azure SQL - passwordless local connection
AZURE_SQL_CONNECTIONSTRING=Server=sql-poc04-<YOUR_SUFFIX>.database.windows.net;Database=sqldb-
poc04;Authentication=ActiveDirectoryDefault;Encrypt=yes;TrustServerCertificate=no;

# Validation
CRITICAL_CONFIDENCE_THRESHOLD=0.70
AMOUNT_TOLERANCE=0.05
ENABLE_SQL_LOAD=true
```

Important Final-Project Clarifications

Storage authentication

Earlier notes mentioned `AZURE_STORAGE_CONNECTION_STRING`. The final project uses `AZURE_STORAGE_ACCOUNT_NAME` with `DefaultAzureCredential`, so a storage connection string is not required for the local batch.

SQL table names

Some intermediate notes say `invoice_line_item`. The final schema uses `dbo.invoice_line`.

Quarantine/audit

Some intermediate notes mention `dbo.processing_audit_log`. The final schema does not create it. Failure reasons are stored as JSON under `documents/failed/` and printed in batch output.

Why this matters

Use the final code/schema names above when reproducing the POC. These differences explain several confusing verification steps in the raw setup notes.

Create the Azure SQL Schema Before run_batch

With `ENABLE_SQL_LOAD=true`, the batch calls SQL before writing successful processed markers. Therefore create the schema first.

SQL

```
-- sql/create_tables.sql creates:
dbo.invoice_header
  - invoice_key (identity PK)
  - source_hash (unique)
  - invoice_number, invoice_date
  - supplier_name, customer_name
  - currency, subtotal, tax, total
  - source_blob, processed_at

dbo.invoice_line
  - invoice_line_key (identity PK)
  - invoice_key (FK)
  - line_number, description
  - quantity, unit_price, amount
```

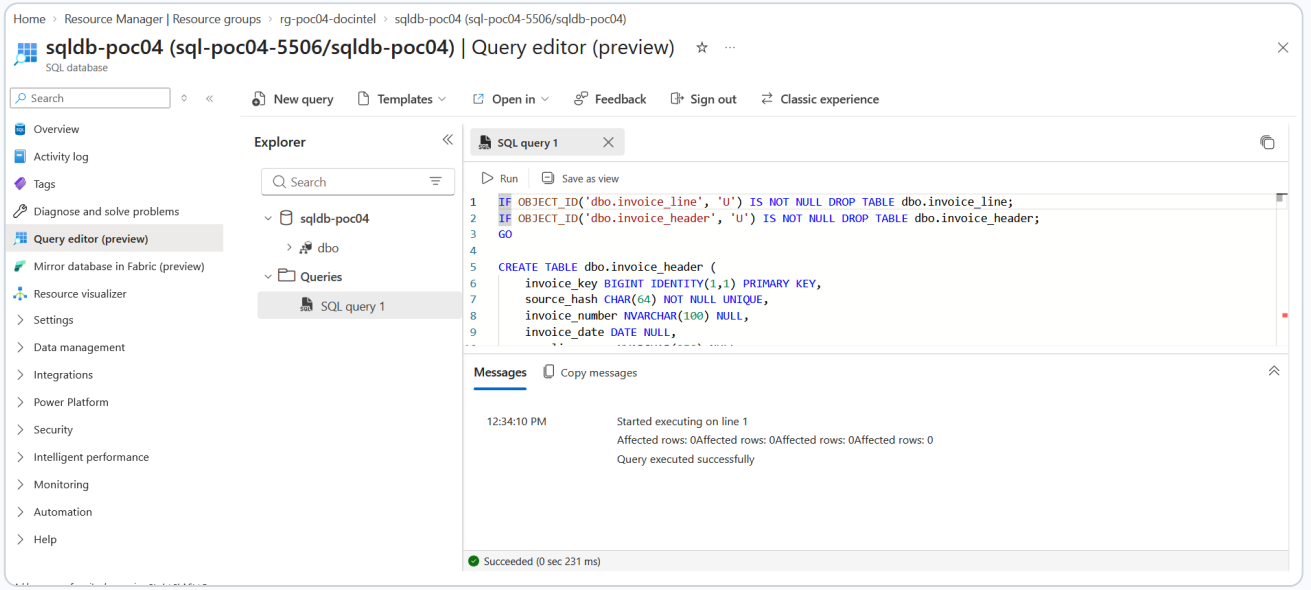
Idempotency guard

The unique constraint on `source_hash` is one of the duplicate-protection layers.

Run create_tables.sql and Confirm the Tables

Portal method

- Open the SQL database `sqldb-poc04`.
- Open Query editor (preview).
- Authenticate with Microsoft Entra ID or the SQL provisioning login.
- Paste the full contents of `sql/create_tables.sql`.
- Click **Run**.
- Confirm `dbo.invoice_header` and `dbo.invoice_line` exist.



Successful table creation in Azure SQL Query Editor.

Which Python Files You Run Directly - and Which You Do Not

File	How to use it	Why
`src/validate_invoice.py`	`python -m pytest tests/test_validation.py`	Library module with validation functions; no CLI `main()`.
`src/upload_samples.py`	`python -m src.upload_samples`	Runnable CLI module that uploads PDFs/images to `incoming/`.
`src/run_batch.py`	`python -m src.run_batch`	Main local batch runner.
`src/pipeline.py`	Do not run directly	Library class used by `run_batch` and the Function App.
`src/extract_invoice.py`	Do not run directly	Extraction helper called by the pipeline.
`src/sql_loader.py`	Do not run directly	SQL helper called by the pipeline.

Correct execution pattern

Running `python src\upload_samples.py` can cause `attempted relative import with no known parent package`. Use the `-m` module form from the project root.

What validate_invoice.py Checks

Hard validation errors

- `invoice_number` is required.
- `total` must be greater than 0.
- If all line amounts are calculable, their sum must match `subtotal` within tolerance.
- `subtotal + tax` must match `total` within tolerance.
- Any critical field confidence below the configured threshold causes failure.

Critical confidence fields

- invoice_number
- invoice_date
- supplier_name
- subtotal
- total

Defaults

DOTENV

```
CRITICAL_CONFIDENCE_THRESHOLD=0.70
AMOUNT_TOLERANCE=0.05
```

Beginner note

Missing confidence scores generate warnings; scores below the threshold generate a validation error.

Upload the Six Sample Invoices

With `.env` configured and `az login` active:

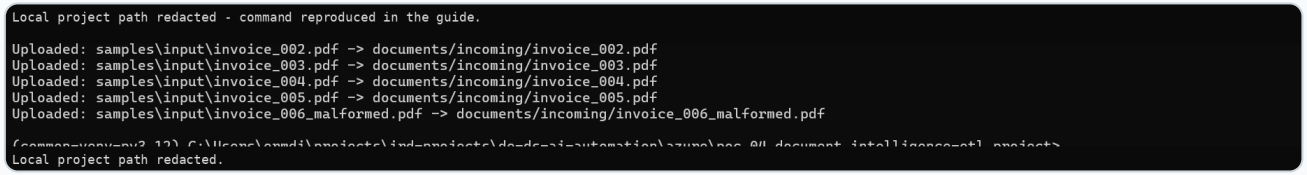
```powershell
python -m src.upload_samples
```

The module scans `samples/input` for PDF/PNG/JPG/JPEG files and uploads each file to:

```
documents/incoming/<filename>
```

Expected result

Expected: six `Uploaded:` lines for invoice_001 through invoice_006_malformed.



Sample invoices uploaded successfully to the documents/incoming prefix.

Run the Batch ETL - What Happens Internally

Run every input under `incoming/`:

POWERSHELL

```
python -m src.run_batch
```

1. `storage_client.py` lists and downloads the blob.
2. `pipeline.py` computes SHA-256 and checks processed/failed markers and SQL duplicates.
3. `extract_invoice.py` calls Document Intelligence using the prebuilt invoice model.
4. Raw analysis JSON is stored under `processed/raw/`.
5. `validate_invoice.py` checks fields, confidence and totals.
6. Valid: SQL upsert + normalized JSON under `processed/`. Invalid: failure JSON under `failed/`.

USEFUL VARIANTS

```
python -m src.run_batch --blob incoming/invoice_001.pdf
python -m src.run_batch --prefix incoming/
```

Challenge: run_batch Failed with Invalid object name

Observed error

`Invalid object name 'dbo.invoice_header'` appeared for each invoice.

Root cause

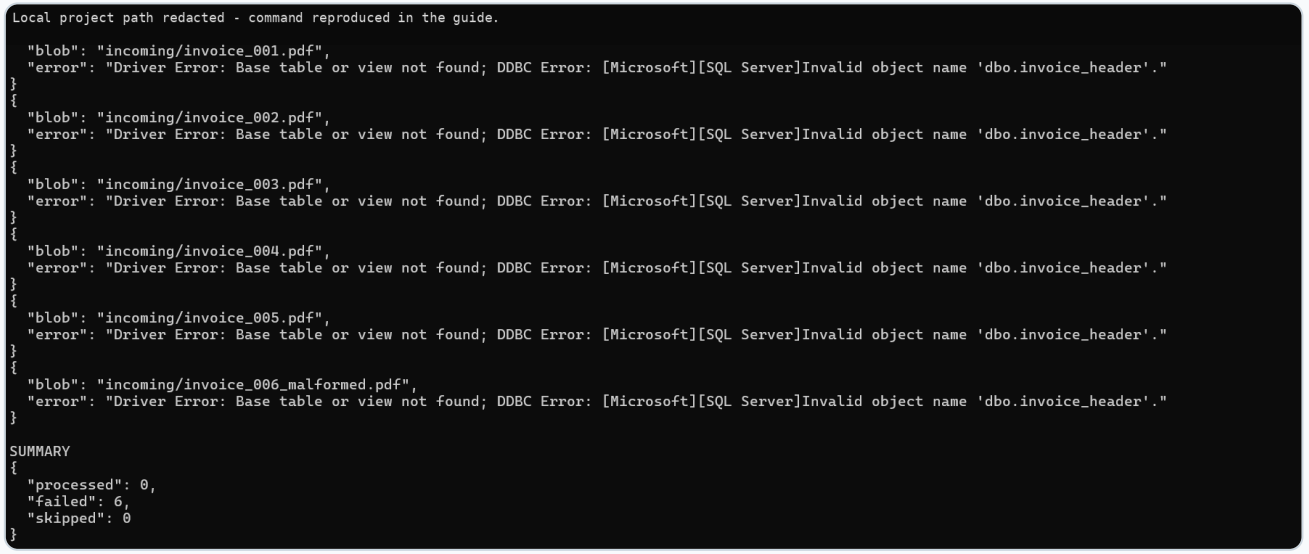
Azure SQL authentication/connection was already working, but the ETL attempted to query/write `dbo.invoice_header` before `sql/create_tables.sql` had been executed.

Meaning

This is a schema-initialization ordering problem, not a Document Intelligence extraction failure.

Beginner note

The batch summary showed zero processed rows because every item failed at the SQL stage.



First batch run: SQL table missing error.

Fix: Initialize SQL, Then Re-run the Batch

Preferred visual fix

- Open Azure SQL Query Editor.
- Run the full `sql/create_tables.sql`.
- Confirm both tables exist.
- Re-run the batch.

POWERSHELL

```
python -m src.run_batch
```

Alternative local Python execution

If your SQL driver can execute the project connection string, the raw POC used this one-liner:

COMMAND

```
python -c "import pyodbc, os; from dotenv
import load_dotenv; load_dotenv(); conn =
pyodbc.connect(os.getenv('AZURE_SQL_CONNECTIONS
TRING')); sql = open('sql/
create_tables.sql').read();
[conn.cursor().execute(stmt) for stmt in
sql.split('GO') if stmt.strip()];
conn.commit(); print('Tables created
successfully!')"
```

Beginner note

If local ODBC/driver authentication becomes a distraction, use Azure Portal Query Editor instead. The completed POC successfully used the portal for visual verification.

Re-run Result - Processing Succeeds

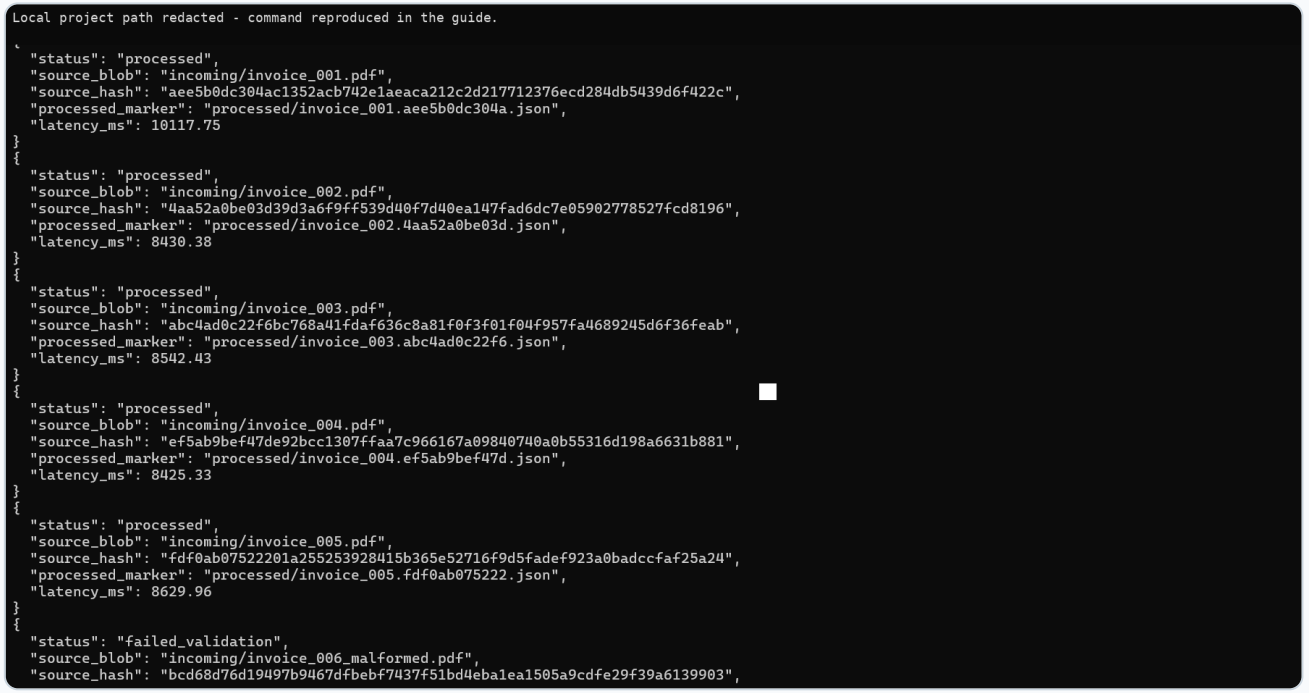
After the SQL schema exists, run:

```powershell
python -m src.run_batch
```

For successful invoices, the output includes:

- `status: processed`
- `source_blob`
- `source_hash`
- `processed_marker`
- `latency_ms`

The malformed sample returns a validation failure and is routed to the failed marker rather than inserted into SQL.



Successful batch output after creating the SQL tables.

Understand the Blob Output Paths

Raw extraction

`documents/processed/raw/<stem>.<hash12>.raw.json`

Preserves the Document Intelligence response for troubleshooting and traceability.

Validated output

`documents/processed/<stem>.<hash12>.json`

Normalized invoice + source metadata + validation + telemetry.

Quarantine

`documents/failed/<stem>.<hash12>.failure.json`

Contains status, source information, failure reasons and warnings.

Beginner note

These are Blob prefixes, not pre-created physical folders. The application creates files beneath them as processing occurs.

Normalized Output JSON - What to Inspect

The validated JSON written under `documents/processed/` includes the normalized business fields plus source, validation and telemetry metadata.

NORMALIZED JSON HIGHLIGHTS

```
source/
  blob_name
  sha256
  processed_at_utc
validation/
  is_valid
  errors[]
  warnings[]
  low_confidence_fields[]
telemetry/
  processing_latency_ms
  low_confidence_field_count
```

Expected result

For invalid documents, the failure JSON under `documents/failed/` includes `failure_reason` and warnings. This storage-based record is the final project's quarantine/audit mechanism.

Verify SQL - Why the Query Editor Showed 0 Rows

WORKFLOW

```
python -m src.run_batch
# then open sql/verify.sql in Query
Editor
```

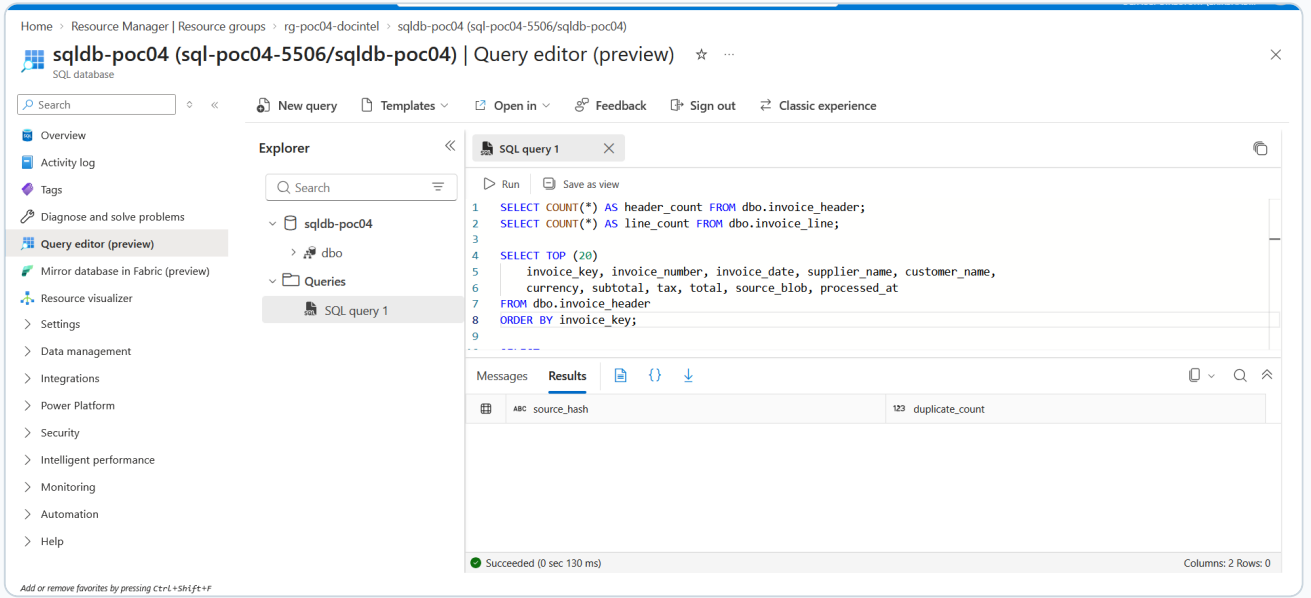
`sql/verify.sql` contains several SELECT statements. The final statement checks only for duplicate source hashes:

SQL

```
SELECT source_hash, COUNT(*) AS
duplicate_count
FROM dbo.invoice_header
GROUP BY source_hash
HAVING COUNT(*) > 1;
```

The confusing 0-row result

0 rows is correct for this last query when no duplicates exist. Run/highlight the earlier SELECT statements individually to see header and line data.



Query Editor showing the duplicate check returning 0 rows - expected when idempotency is working.

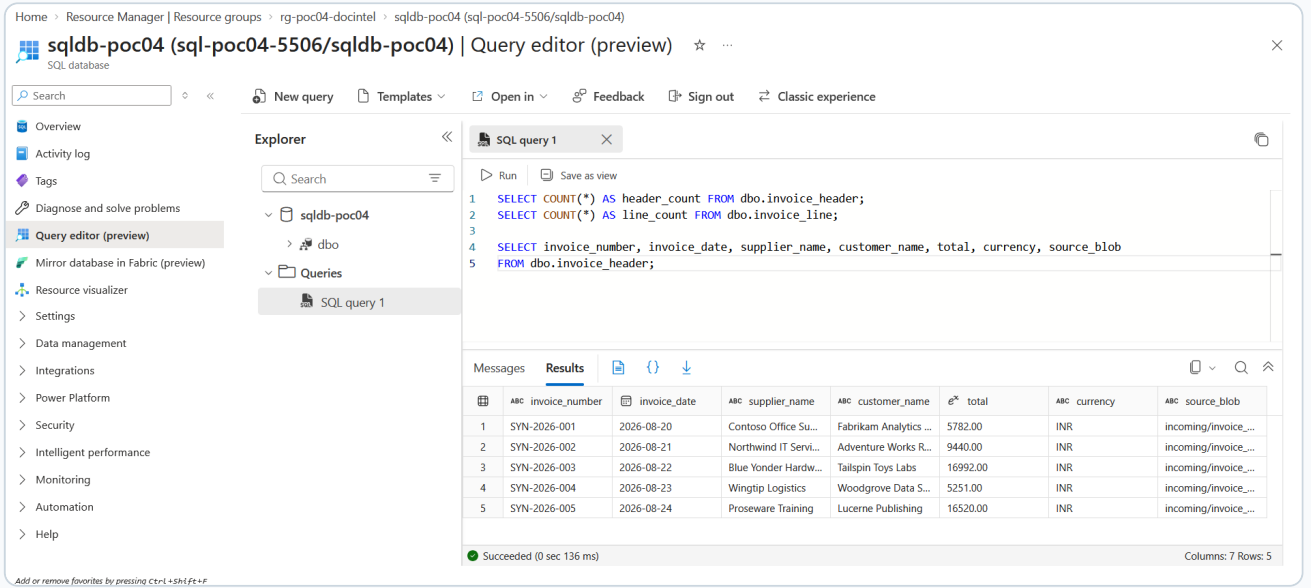
Verify Curated Invoice Headers

Run this query by itself:

```sql
SELECT TOP (20)
    invoice_key, invoice_number,
invoice_date,
    supplier_name, customer_name,
    currency, subtotal, tax, total,
source_blob
FROM dbo.invoice_header
ORDER BY invoice_key;
```

Expected result

Expected for this POC: five valid header rows from invoice_001 through invoice_005. The malformed sample should not appear as a valid SQL header.



Azure SQL invoice_header rows after successful processing.

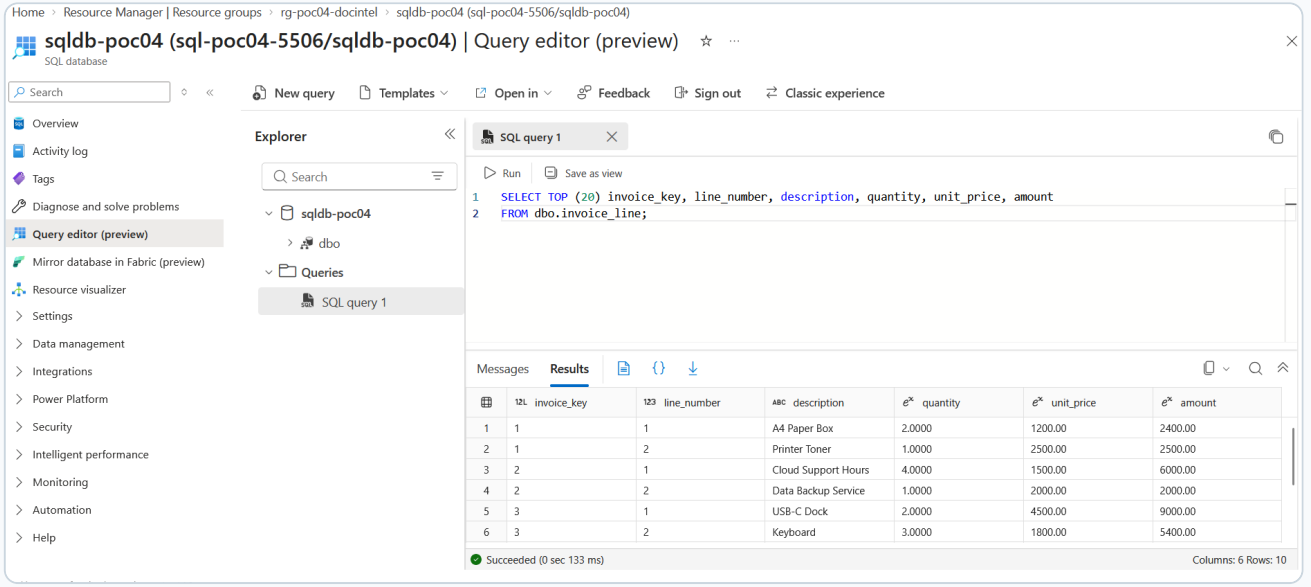
Verify Invoice Line Items

Inspect the child table:

```sql
SELECT TOP (20)
    invoice_key, line_number,
description,
    quantity, unit_price, amount
FROM dbo.invoice_line
ORDER BY invoice_key, line_number;
```

Each line references its parent `invoice_header.invoice_key`.

> **Expected result**
>
> This confirms the extraction was flattened into relational header/line form rather than stored only as JSON.



Parsed invoice line items stored in dbo.invoice_line.

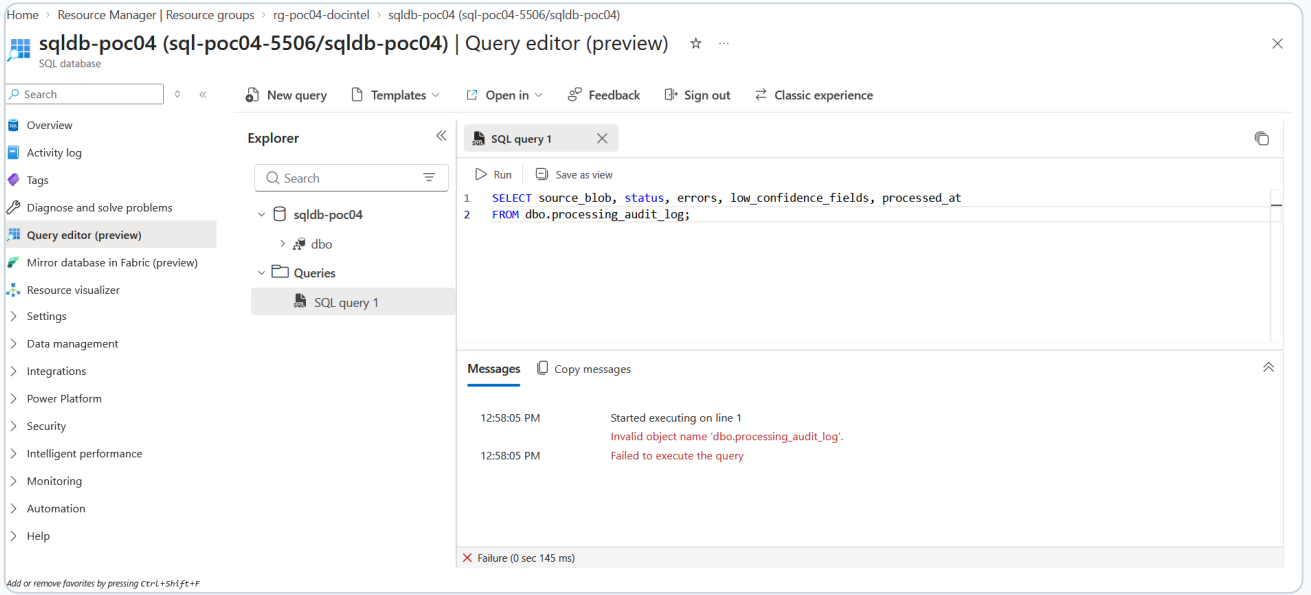
Final SQL Integrity Check

Use simple checks one at a time:

```sql
SELECT COUNT(*) AS header_count FROM
dbo.invoice_header;
SELECT COUNT(*) AS line_count FROM
dbo.invoice_line;

SELECT source_hash, COUNT(*) AS
duplicate_count
FROM dbo.invoice_header
GROUP BY source_hash
HAVING COUNT(*) > 1;
```

Expected result

The duplicate query should return **0 rows**. This screenshot shows the final Query Editor verification state.



Final verification: no duplicate source_hash rows.

Reprocessing Test - Prove Idempotency

Run the same incoming set again without changing the files:

```powershell
python -m src.run_batch
```

Possible duplicate statuses

- `skipped_duplicate` - normalized processed marker already exists in Blob Storage.
- `skipped_duplicate_failed` - failure marker already exists for the same SHA-256.
- `skipped_duplicate_sql` - SQL already contains the same source hash.

Why it works

- The pipeline hashes the exact file bytes with SHA-256.
- Blob output filenames include the first 12 characters of that hash.
- `invoice_header.source_hash` has a UNIQUE constraint.

> **Expected result**
> Pass condition: running the same blob again does not create another SQL header/line set.

Troubleshooting Playbook - Issues 1 to 4

Issue	Root cause	Fix
1. ModuleNotFoundError: src when running standalone pytest	Project root not on Python import path in that invocation.	`python -m pytest tests/test_validation.py` or `pytest.ini` with `[pytest] pythonpath = .`.
2. attempted relative import with no known parent package	Running `python src\upload_samples.py` directly loses package context.	Run from root: `python -m src.upload_samples`.
3. Invalid object name dbo.invoice_header	SQL schema not created before the batch.	Run `sql/create_tables.sql`, verify both tables, then rerun `python -m src.run_batch`.
4. SQL driver / DSN not found	Local driver setup does not match the chosen SQL connection method.	Install/configure a supported SQL Server driver or use Azure Portal Query Editor for schema/ verification.

Troubleshooting Playbook - Issues 5 to 8

Issue	Root cause	Fix
5. Windows logins are not supported (40607)	A legacy/default local SQL driver can fall back to Windows authentication instead of the intended Entra authentication.	Use the intended modern driver/authentication path, or verify through Azure Portal Query Editor.
6. Query Editor shows 0 rows after verify.sql	The final duplicate query legitimately returns no rows.	Run each SELECT individually. Use `SELECT * FROM dbo.invoice_header` to view data.
7. processing_audit_log table not found	The final project schema intentionally contains only header + line tables.	Inspect `documents/failed/*.failure.json` and `src.run_batch` output for quarantine reasons.
8. Azure SQL firewall blocks connection	Current public IP is not allowed.	Add only your current client IP in Azure SQL networking/firewall, then retry.

Additional Azure Authentication / Storage Checks

DefaultAzureCredential failed

```powershell
az login
az account show
```

- Blob access: your user needs **Storage Blob Data Contributor**.
- Document Intelligence with identity: needs **Cognitive Services User**.
- For the first local DI run, the project can use the API key from local `.env`.

Storage AuthorizationPermissionMismatch

Assign the signed-in identity **Storage Blob Data Contributor** and allow time for RBAC propagation.

Document Intelligence 401/403

Check endpoint and credential selection. For key auth, confirm `DOCUMENTINTELLIGENCE_API_KEY` is loaded and contains no accidental spaces.

InvalidContent

Ensure the source PDF/image is supported, non-empty and not corrupted.

Optional - Deploy the Event-Driven Function

Grant SQL identity

- Open `sql/create_function_identity_user.sql`.
- Replace `<FUNCTION_APP_NAME>` with your Function App name.
- Run the script as an Azure SQL Microsoft Entra administrator.

Test locally

POWERSHELL

```
func start
```

Publish

POWERSHELL

```
func azure functionapp publish func-poc04-di-<YOUR_SUFFIX>
```

Cloud behavior

- Blob trigger listens to the incoming path.
- Function uses Managed Identity for storage/Document Intelligence.
- Function SQL connection uses `Authentication=ActiveDirectoryMSI` in the provisioning script.
- Processed/failed output prefixes do not match the incoming trigger path, avoiding output retriggers.

Observability and What to Monitor

Application Insights / Function Monitor

- Function invocations.
- Log entry indicating the invoice trigger received a blob.
- Pipeline result/status.
- Exceptions and failed invocations.

Pipeline telemetry already stored in output JSON

```json
telemetry:
  processing_latency_ms
  low_confidence_field_count
```

POC metrics to reason about

- documents processed
- success/failure count
- processing latency
- low-confidence field count

Cleanup and Cost Control

When testing is complete, delete the cloud resources so the POC does not continue to incur cost:

COMMAND PROMPT / POWERSHELL

```
powershell -ExecutionPolicy Bypass -File scripts\99_cleanup.ps1
```

When the script asks for confirmation, type the required confirmation value.

Local cleanup

COMMAND PROMPT

```
del .env
del .poc04-resources.txt
deactivate
```

> **Important**
>
> Before deleting `.env`, make sure you no longer need the local configuration. Never commit it. If any secret was exposed in notes/screenshots during the POC, rotate that secret before future use.

Command Cheat Sheet - Setup and Configuration

COMMAND PROMPT / POWERSHELL

```
# Login and subscription
az login
az account show

# Python environment
python -m venv .venv
.venv\Scripts\activate.bat
python -m pip install --upgrade pip
pip install -r requirements.txt

# Unit tests
python -m pytest tests/test_validation.py

# Provision Azure
powershell -ExecutionPolicy Bypass -File scripts\01_create_azure_resources.ps1
powershell -ExecutionPolicy Bypass -File scripts\02_show_resource_values.ps1

# Create local env
copy .env.example .env
notepad .env

# Retrieve local DI key
az keyvault secret show --vault-name kv-poc04-<SUFFIX> --name document-intelligence-key --query value
-o tsv
```

Command Cheat Sheet - Run, Verify and Re-run

COMMAND PROMPT / POWERSHELL

```
# Optional: regenerate synthetic invoices
python scripts/generate_sample_invoices.py

# Upload samples
python -m src.upload_samples

# Run all incoming files
python -m src.run_batch

# Run a single blob
python -m src.run_batch --blob incoming/invoice_001.pdf

# Run a prefix explicitly
python -m src.run_batch --prefix incoming/

# Optional Function
func start
func azure functionapp publish func-poc04-di-<SUFFIX>

# Cleanup
powershell -ExecutionPolicy Bypass -File scripts\99_cleanup.ps1
```

Final Completion Checklist

- [ ] Python dependencies installed and tests pass.
- [ ] `documents` container exists and your user has Blob data access.
- [ ] Local `.env` contains account name, DI endpoint/key and SQL connection string.
- [ ] Six sample invoices uploaded to `documents/incoming/`.
- [ ] Malformed invoice appears under `documents/failed/` with reasons.
- [ ] Re-running the same blobs does not create duplicates.
- [ ] Secrets are not in Git/documentation.
- [ ] Resource Group contains ADLS, Document Intelligence, Key Vault, SQL and optional Function resources.
- [ ] One sample invoice was manually analyzed with prebuilt invoice.
- [ ] `dbo.invoice_header` and `dbo.invoice_line` exist.
- [ ] Valid invoices appear in Azure SQL.
- [ ] Raw DI JSON appears under `documents/processed/raw/`.
- [ ] Duplicate SQL query returns 0 rows.
- [ ] Resource Group is deleted after the POC if no longer needed.

Quick Interview Revision

OCR vs Document Intelligence?

OCR gives text/layout; Document Intelligence adds document-specific structured fields, line items and confidence information.

Why keep raw + normalized JSON?

Raw output supports debugging/reprocessing; normalized output gives a stable schema for downstream systems.

How do confidence thresholds affect automation?

They decide which extracted documents can proceed automatically and which should be quarantined/reviewed.

How are duplicates prevented?

SHA-256 file hash, Blob processed/failed markers, and a unique SQL `source_hash`.

What is the quarantine path?

`documents/failed/*.failure.json` containing the failure reason and available normalized data.

How would you scale?

Use event-driven ingestion, durable retry/queue patterns, identity-based authentication, monitoring, partitioned storage and scalable curated targets.

> **Beginner note**
> For your CV, use the project bullets only after you can reproduce the setup, explain the validation rules, show the SQL output and explain the errors you solved.